SEO PACKAGE

1. SEO Title

SQL Indexes Explained: How Databases Find 1 Row in a Million in ~20 Steps

2. SEO Subtitle

Understand the B-tree magic behind database indexes, when to use them, and the hidden costs most beginners miss.

3. Featured Quote

> "A full table scan reads everything. An index reads a shortcut. The difference between the two is the difference between flipping through a million phone book entries and opening straight to the right page."

4. Meta Description

Learn how SQL indexes turn a million-row full table scan into ~20 steps using a B-tree, plus when to index, when not to, and the costs nobody warns you about.

5. URL Slug

`sql-indexes-explained-btree-query-performance`

6. Keywords

SQL indexes, database index, B-tree index, query performance, full table scan, composite index, CREATE INDEX, SQL optimization, index vs full scan, logarithmic search, EXPLAIN query plan, database performance tuning, indexing best practices, when to use indexes, slow SQL queries, index write cost, WHERE clause optimization, ORDER BY index, JOIN performance, PostgreSQL index

7. Tags

`SQL` `Database Indexes` `B-Tree` `Query Optimization` `Backend Engineering` `PostgreSQL` `MySQL` `Performance Tuning` `Data Structures` `Software Engineering` `Learn SQL` `Coding Interview` `Databases` `EXPLAIN`

The Index Trick: How SQL Finds One Row in a Million Without Reading Them All

8. Introduction

Every backend engineer eventually meets the same wall: a query that ran instantly on your laptop with a thousand test rows grinds to a halt in production with a few million. Nothing in your code changed. The table just got bigger. And suddenly a page that loaded in 40 milliseconds takes eight seconds.

This is one of the most practical, high-leverage topics in all of software engineering, and it shows up constantly in system design interviews, backend coding interviews, and real on-call incidents. Interviewers ask about indexes because the answer reveals whether you understand what the database is *actually doing* underneath your `SELECT`, or whether you just write SQL and hope. They want to know: Do you understand the difference between linear and logarithmic search? Can you explain a B-tree? Do you know that indexes aren't free?

If you've ever wondered why one query is instant and a nearly identical one crawls, this is the lesson that connects the dots. By the end you'll know exactly what an index is, why it's so fast, the trade-off it quietly makes, and how to prove it's working instead of guessing. This is core knowledge for anyone working with databases, and it separates developers who *use* SQL from developers who *understand* it.

9. Problem Overview

Here's the problem stated plainly.

You have a table with a million rows, users, orders, events, anything, and you want to find one specific row:

```
SELECT * FROM users WHERE email = 'kai@example.com';
```

With no help, the database has exactly one strategy: start at the first row, check it, move to the next, check it, and keep going until it either finds a match or reaches the end. This is called a **full table scan**. In the worst case, the row you want is the last one, so the engine reads all one million rows just to answer a question about one.

The core challenge: **how do we find a single row without looking at every row?**

Think of a phone book with a million names in random order. To find one person, you'd have to read every entry, there's no shortcut, because there's no order to exploit. Now imagine the same phone book sorted alphabetically. You don't read it top to bottom. You open to the middle, decide "earlier or later," and jump. That ordering is the entire secret. An **index** is the database's way of keeping a sorted shortcut to your data so it can jump instead of crawl.

10. Example

Let's make the numbers concrete, because the scale is the whole point.

Table size	Full table scan (linear)	Index search (logarithmic)
1,000 rows	up to 1,000 checks	~10 checks
100,000 rows	up to 100,000 checks	~17 checks
1,000,000 rows	up to 1,000,000 checks	~20 checks
1,000,000,000 rows	up to 1,000,000,000 checks	~30 checks

Read that bottom row twice. Going from a million rows to a **billion** — a thousand times more data — the index goes from ~20 checks to ~30. Ten extra steps to handle a thousand times the data.

That's because each index step **throws away half** of what's left. Halving a million rows takes about 20 steps ($2^{20} \approx 1{,}048{,}576$). The full scan, meanwhile, doubles its work every time you double the table. This gap isn't a small optimization. At a million rows it's roughly **fifty thousand times less work** for the same result on the same machine.

11. Intuition

Here's how an experienced engineer arrives at indexes, without memorizing anything.

Start with the pain: reading every row is slow, and it gets *linearly* slower as the table grows. So the instinct is, "I don't want to look at everything. I want to skip." But you can only skip if you can make a decision that eliminates a chunk of data without examining it.

What lets you eliminate data you haven't looked at? **Order.** If values are sorted, then checking one value tells you something about a whole range of other values. Look at the middle value: if your target

is bigger, everything to the left is irrelevant, gone, no need to read it. That single comparison just deleted half the search space.

Repeat that idea and you get **binary search**: jump to the middle, compare, discard half, repeat. Each step halves what remains, which is exactly why the step count grows logarithmically instead of linearly.

The last leap is realizing the *table itself* usually isn't sorted (and can't be sorted by every column at once). So instead of reordering your data, the database keeps a **separate, small, sorted structure** that maps each value to where its row physically lives, like the index at the back of a textbook pointing "photosynthesis → page 212." You don't reorganize the book. You add a signpost. That signpost is the index, and in practice it's stored as a **B-tree**.

12. Brute Force Solution

Idea

The brute force approach is the full table scan: no index, just read every row and test the condition.

Algorithm

1. Start at the first row.
2. Check whether it matches the `WHERE` condition.
3. If it matches, keep it (and, if not unique, continue).
4. Move to the next row.
5. Repeat until the table ends.

Advantages

- **Zero setup.** No extra structure to build or maintain.
- **No write penalty.** Inserts, updates, and deletes stay as cheap as possible.
- **Fine for small tables.** On a few hundred rows, a scan is instant — sometimes *faster* than an index, because there's no tree to traverse and no extra disk pages to read.

Disadvantages

- **Scales linearly.** Double the rows, double the time. This is the killer.
- **Wasteful for selective lookups.** Reading a million rows to return one is enormous overhead.
- **Hurts joins and sorting too**, not just simple filters.

Complexity

- **Time:** $O(n)$ — every row may be read.
- **Space:** $O(1)$ — no extra storage beyond the table itself.

13. Optimal Solution

The optimal approach is to **create an index** on the column you filter by, so the engine can binary-search a sorted structure instead of scanning.

Under the hood, most databases build the index as a **B-tree** (balanced tree). Picture it as a signpost tree of sorted values:

```
            [ M ]
           /     \
      [ D | H ]   [ R | W ]
      /   |   \     |    \
...   rows  ...  rows   ...
```

You start at the top (root) node, follow the branch whose range contains your value, drop to the next level, follow again, and after a handful of hops you land on a leaf node that points directly at the row. Because the tree is balanced, every lookup takes the same small number of hops, roughly the logarithm of the row count.

Where indexes earn their keep:

Operation	Why the index helps
Filtering (`WHERE`)	Jump straight to matching values instead of scanning.
Sorting (`ORDER BY`)	Data is already stored sorted, so the engine can skip the sort step entirely.
Joining (`JOIN`)	Matching rows across tables becomes a fast lookup instead of a hunt.

Creating one is a single line:

```
CREATE INDEX idx_users_email ON users (email);
```

From that moment, the database quietly maintains a sorted copy of the `email` column and reaches for it every time you filter, sort, or join on `email`.

But there's an honest catch. An index is a second copy of that column that must stay in sync with the table. So every `INSERT`, `UPDATE`, and `DELETE` now does extra bookkeeping to update the tree, and the index consumes disk space. Indexes make **reads faster but writes a little slower**. It's a trade, not free candy.

14. Python Code

Databases do this in C, but the intuition is easiest to *feel* in a few lines of Python. Below, the full scan and the index search return the same answer, the second just does far less work.

```
from bisect import bisect_left


def full_table_scan(rows, target_email):
    """Brute force: check every row until we find a match. O(n)."""
    for row in rows:
        if row["email"] == target_email:
            return row
    return None


def build_index(rows):
    """Build a sorted 'index': (email, row) pairs sorted by email. O(n log n) once."""
    return sorted(((row["email"], row) for row in rows), key=lambda pair: pair[0])


def index_search(index, target_email):
    """Binary search the sorted index — the B-tree idea, flattened. O(log n)."""
    keys = [email for email, _ in index]
    pos = bisect_left(keys, target_email)
    if pos < len(index) and index[pos][0] == target_email:
        return index[pos][1]
    return None


if __name__ == "__main__":
    users = [{"id": i, "email": f"user{i}@example.com"} for i in range(1_000_000)]
    target = "user823456@example.com"

    # Brute force: may walk through hundreds of thousands of rows.
    assert full_table_scan(users, target)["id"] == 823456

    # Indexed: builds the sorted structure once, then jumps in ~20 comparisons.
    idx = build_index(users)
    assert index_search(idx, target)["id"] == 823456

    # A value that isn't there returns None from both paths.
    assert full_table_scan(users, "ghost@example.com") is None
    assert index_search(idx, "ghost@example.com") is None
    print("All checks passed.")
```

15. Code Walkthrough

- `full_table_scan` is the brute force. It loops through every row in order and compares. Simple, correct, and slow, exactly what the database does without an index.
- `build_index` mirrors what `CREATE INDEX` does: it produces a **sorted** list of `(email, row)` pairs. This sort is the one-time cost you pay so that all future lookups are cheap. Real databases pay this cost incrementally as rows are inserted, which is why writes get slightly slower.
- `index_search` is binary search via Python's `bisect_left`. `bisect_left` finds where the target *would* go in the sorted keys in $O(\log n)$ comparisons, the flat-list version of walking down a B-tree. We then confirm the value at that position actually equals the target (otherwise it's a "not found").
- The `__main__` **self-check** proves both paths agree, including the miss case (`ghost@example.com` → `None`). If either the scan or the binary search ever disagreed, an `assert` would fail immediately.

One honest note for realism: in this toy version, `index_search` rebuilds `keys` on every call, which itself is $O(n)$. A real B-tree stores the keys *in* the tree nodes, so no rebuild happens, the lookup is genuinely $O(\log n)$ end to end. The point here is the *shape* of the idea, not a production engine.

16. Dry Run

Let's search a tiny sorted index of 8 emails for `"g@x.com"` using binary search. Positions 0–7:

```
[ a@x  b@x  c@x  d@x  e@x  f@x  g@x  h@x ]
   0    1    2    3    4    5    6    7
```

- **Step 1:** Range is 0–7. Middle is position 3 (`d@x`). Is `g@x` > `d@x`? Yes. Discard 0–3. Remaining: 4–7.
- **Step 2:** Range is 4–7. Middle is position 5 (`f@x`). Is `g@x` > `f@x`? Yes. Discard 4–5. Remaining: 6–7.
- **Step 3:** Range is 6–7. Middle is position 6 (`g@x`). Match! Return the row at position 6.

Three comparisons for eight rows ($2^3 = 8$). A full scan would have taken up to seven. The gap looks small here, but it's the *same three-versus-seven idea* that becomes twenty-versus-a-million at scale.

17. Complexity Analysis

Full table scan - **Time: $O(n)$** — in the worst case the matching row is last (or absent), so all n rows are read. - **Space: $O(1)$** — no extra structure.

Index (B-tree) search - **Time: $O(\log n)$** per lookup — each level of the balanced tree eliminates a large fraction of remaining rows, so height grows logarithmically with row count. This is why a million rows resolves in ~20 steps. - **Space: $O(n)$** — the index stores an entry per row (per indexed column), which is the disk-space cost. - **Write cost: $O(\log n)$ extra per insert/update/delete** — the tree must be updated and kept balanced, which is the "writes get slower" trade.

Why these are correct: a scan has no way to skip rows, so its work is proportional to table size. A B-tree makes a branching decision at each level that discards a constant fraction of candidates, and the number of times you can divide n before reaching 1 is $\log n$.

18. Alternative Solutions

Indexes aren't the only tool, and a strong engineer matches the tool to how the table is *actually used*.

Approach	Best when	Trade-off
B-tree index	Read-heavy tables, selective lookups, sorting, joins	Slows writes, uses disk
Composite index	You filter on two columns together (e.g. `customer_id, created_at`)	Only helps if the query uses the **leftmost** column(s)
Caching	The exact same query runs over and over	Stale data risk; cache invalidation is hard
Keeping the table lean / partitioning	Very write-heavy tables where indexes hurt too much	More operational complexity
Hash index	Exact-equality lookups only (`=`), no ranges	Useless for `>`, `<`, `BETWEEN`, or `ORDER BY`

A **composite index** is worth a special mention. Think of sorting a class first by grade, then by name within each grade:

```
CREATE INDEX idx_orders_customer_date ON orders (customer_id, created_at);
```

This flies when you filter by `customer_id`, or by `customer_id` *and* `created_at` together. But a query that filters by `created_at` **alone** can't use it — the list was sorted by `customer_id` first, so `created_at` values are scattered across the whole tree. Column order in a composite index is a decision, not a formality.

19. Edge Cases

- **Empty table:** Both approaches return nothing instantly. The index adds pure overhead here.

- **Small tables (a few hundred rows):** A full scan can *beat* an index — traversing the tree and reading extra pages isn't worth it. Don't index tiny lookup tables reflexively.
- **Duplicate values:** A non-unique index still works, but returns many rows; the engine walks a run of matching leaves. If the column *should* be unique, use a **unique index** to enforce it and speed exact lookups.
- **Low-cardinality columns** (e.g. a boolean `is_active`, or `status` with 3 values): an index barely helps, since matching "half the table" still means reading half the table. The optimizer may correctly ignore it.
- **NULLs:** How NULLs are stored/searched in an index varies by database — don't assume `WHERE col IS NULL` uses your index the way `WHERE col = x` does.
- **Very large / high-write tables:** the write penalty of many indexes can dominate; sometimes fewer indexes is faster overall.

20. Common Mistakes

1. **Indexing everything "just in case."** Every index taxes every write and eats disk. Unused indexes are pure cost. Index on purpose, not on impulse.
2. **Wrapping the indexed column in a function.** `WHERE LOWER(email) = 'kai@example.com'` silently **disables** the index, because the engine can't match `LOWER(email)` against the raw sorted values. Store the column normalized, or create a function/expression index instead. (This one costs people entire afternoons of debugging a "randomly slow" query.)
3. **Ignoring column order in composite indexes.** An index on `(a, b)` does **not** help a query that filters on `b` alone.
4. **Guessing instead of checking.** Never assume an index is being used. Run `EXPLAIN` — it prints the actual plan and tells you whether it's a `Seq Scan` (full scan) or an `Index Scan`.
5. **Indexing low-cardinality columns.** An index on a boolean rarely helps; the engine still touches a huge fraction of rows.
6. **Forgetting the write cost entirely.** Piling indexes on a write-heavy table can make the whole system slower, not faster.
7. **Type mismatches.** Comparing an indexed integer column against a string literal can force an implicit cast that skips the index.

21. Interview Questions

- **What's the difference between a clustered and a non-clustered index?** (Clustered defines the physical row order; a table has at most one. Non-clustered is a separate structure

pointing at rows.)

- **Why is a B-tree preferred over a hash index for most general-purpose indexing?** (B-trees support range queries and ordering; hash indexes only do equality.)
- **You added an index but the query is still slow — how do you diagnose it?** (Run `EXPLAIN` / `EXPLAIN ANALYZE`; check for functions on the column, type mismatches, low selectivity, or wrong composite order.)
- **When would you deliberately *not* add an index?** (Small tables, write-heavy tables, low-cardinality columns.)
- **How does an index on `(a, b)` behave for queries on `a`, on `b`, and on both?** (Helps `a` and `a+b`; not `b` alone.)
- **What's the trade-off an index makes, in one sentence?** (Faster reads, slower writes, more storage.)
- **How does an index speed up `ORDER BY` without any filtering?** (The data is already stored sorted, so the sort step can be skipped.)

22. Similar Problems

While this is a database concept rather than a single algorithm puzzle, the *mental model* connects directly to classic problems every developer should know:

- **Binary Search (LeetCode 704)** — the exact halving mechanic that makes an index fast. Understand this and B-trees stop feeling like magic.
- **Search in Rotated Sorted Array (LeetCode 33)** — reinforces how *ordering* is the property that lets you discard half the search space.
- **Search a 2D Matrix (LeetCode 74)** — binary search over structured, sorted data, mirroring multi-level index traversal.
- **Design HashMap (LeetCode 706)** — builds intuition for hash indexes and why they only handle equality.
- **Kth Smallest Element in a BST (LeetCode 230)** — a tree of sorted values you navigate by comparison, the same shape as a B-tree lookup.

They're related because each one rests on the same foundation: **impose order, then exploit it to avoid looking at everything.**

23. Key Takeaways

- A **full table scan** reads every row: $O(n)$, and it gets linearly worse as the table grows.

- An **index** keeps a sorted **B-tree** shortcut, turning lookups into $O(\log n)$ — a million rows in ~20 steps.
- Indexes shine on **filtering, sorting, and joining**.
- They aren't free: **reads get faster, writes get slower, and storage grows.**
- **Composite indexes** care about column order — leftmost column wins.
- The classic traps: indexing everything, wrapping columns in functions, and guessing instead of running `EXPLAIN`.
- The one-line rule: **index on purpose, not on impulse — and confirm with `EXPLAIN`.**

24. Watch the Video

Reading builds the model; watching it click is faster. In the video, Kai walks through the phone-book analogy, the B-tree, and, most usefully, shows `EXPLAIN` flipping live from a sequential scan to an index scan the moment the index is added. Seeing that switch happen is the "aha" that makes this stick.

▶️ **Watch Lesson 25 here: {LINK}**

25. About the Series

This is part of **Fun with Learning Technology**, a daily series that takes one real backend, SQL, and software engineering concept and makes it genuinely click, no hand-waving, no jargon dumps, just the intuition an engineer actually uses on the job. Each lesson is short, practical, and built to move you from "I can write it" to "I understand it." If you're preparing for coding interviews or leveling up your day-to-day database skills, following along daily compounds fast.

26. Call To Action

If this made indexes finally make sense:

- 👍 **Like** the video on YouTube — it genuinely helps the channel reach more developers.
- 🔔 **Subscribe** on YouTube so you don't miss the daily lessons.
- 📩 **Subscribe on Substack** for the written deep-dives like this one.
- 💬 **Comment** with your worst "why is this query slow" story — odds are it was a lonely `LOWER()` call.
- 🔁 **Share** it with a teammate who just got paged for a slow query at 3am.

A follow tells Kai to keep these coming.

SOCIAL MEDIA

LinkedIn Post

Your query ran in 40ms on your laptop. In production it takes 8 seconds. Nothing in your code changed — the table just grew from a thousand rows to a million.

This is one of the most practical topics in backend engineering, and it comes down to one idea: the difference between a full table scan and an index.

Without an index, the database does the only thing it can — it reads every row and checks each one. That's linear: double the rows, double the work. A million rows can mean a million checks.

An index changes the game. It keeps a sorted structure (usually a B-tree) that lets the engine binary-search instead of scan. Each step throws away half the remaining data, so a million rows resolve in about 20 steps instead of a million. Same machine, same data, roughly 50,000× less work.

But indexes aren't free, and this is the part beginners miss:

→ Every index is a second copy that must stay in sync → Every INSERT, UPDATE, and DELETE now does extra bookkeeping → They consume disk space

So indexes make reads faster and writes a little slower. It's a trade.

Three traps I see constantly: 1. Indexing everything "just in case" — pure write overhead, no gain 2. Wrapping an indexed column in a function (LOWER(email)) — silently disables the index 3. Guessing instead of running EXPLAIN — which tells you the actual plan

The rule that's served me best: index on purpose, not on impulse — and confirm with EXPLAIN.

What's the slowest query you've debugged, and what fixed it?

SQL #Databases #BackendEngineering #SoftwareEngineering #QueryOptimization

Twitter/X Thread

1/ A query that scans a million rows. An index that finds the answer in ~20 steps. Same data. Same machine. ~50,000× less work. Here's how database indexes actually pull that off 🪄

2/ With no index, the database does a "full table scan." It reads every row and checks each one — like flipping through a million phone book entries in random order. This is linear: double the rows, double the work. It only gets worse as you grow.

3/ An index flips linear into logarithmic. Linear: a million rows → up to a million checks. Logarithmic: a million rows → ~20 checks. Go to a BILLION rows and it's only ~30. Ten extra steps for 1,000× the data.

4/ The trick: keep the values sorted. Jump to the middle → is my target bigger or smaller? → throw away half instantly. Keep halving until one row is left. That's binary search, and that's the magic.

5/ Most indexes are B-trees — a tidy branching tree of sorted values. Start at the top, follow the branch that fits your value, and in a few hops you land on the exact row. A signpost tree that always points the right way.

6/ Creating one is a single line:

CREATE INDEX idx_users_email ON users (email);

From then on, the DB keeps a sorted copy of that column and uses it every time you filter on email.

7/ Indexes shine in three places: • Filtering (WHERE) • Sorting (ORDER BY) — data's already sorted, sort step skipped • Joining (JOIN) — match rows instead of hunting

8/ The honest catch nobody mentions first: An index is a second copy that must stay in sync. Every INSERT/UPDATE/DELETE now has extra bookkeeping, and it eats disk. Reads faster, writes slower. A trade, not free candy.

9/ Two traps that kill indexes silently: • Indexing everything → slows writes for zero gain • LOWER(email) in your WHERE → disables the index, because the engine can't match sorted values anymore (I lost an afternoon to that one.)

10/ One strong opinion: stop indexing by gut. Run EXPLAIN. It shows the real plan — full scan vs index scan. Add the index, run EXPLAIN, watch it flip from sequential scan to index scan. Clean. Index on purpose, not on impulse.

Facebook Post

Ever had a query that was instant on your test data but painfully slow once the table got big? 😅

Here's the whole secret in one picture: imagine a phone book with a million names in random order versus one sorted alphabetically. In the random one, you have to read everything. In the sorted one, you flip straight to the right page. That "sorted shortcut" is exactly what a database index is.

Without it, the database reads every single row (a "full table scan"). With it, it uses a B-tree to keep halving the search — so a million rows takes about 20 steps instead of a million. Same machine, same data, wildly less work.

The catch? Indexes make reads faster but writes a little slower, since every change has to update the index too. So you add them where filtering, sorting, or joining is actually your bottleneck — and you confirm with EXPLAIN instead of guessing.

Full breakdown (with code, diagrams, and the common mistakes) is up now. 👇

Reddit Summary

How databases find 1 row in a million in ~20 steps (indexes, explained by the trade-offs)

Sharing a mental model that clicked for me and might help others working with SQL.

Without an index, a lookup is a full table scan: the engine reads every row and checks it. That's O(n) — a million rows can mean a million checks, and it degrades linearly as the table grows.

An index keeps a sorted structure (usually a B-tree) so the engine can binary-search instead. Each comparison discards half the remaining rows, so it's O(log n): ~20 steps for a million rows, ~30 for a billion. That's the whole reason one query is instant and a similar one crawls.

Where indexes help: WHERE filters, ORDER BY (data's already sorted, so the sort can be skipped), and JOINs.

The part that's easy to miss: an index is a second copy that must stay in sync, so every INSERT/UPDATE/DELETE gets more expensive, and it costs disk. Reads faster, writes slower.

A few things worth knowing: - Wrapping an indexed column in a function (e.g. `LOWER(email)`) usually disables the index. - Composite indexes only help if your query uses the leftmost column(s). An index on `(a, b)` won't help a query filtering on `b` alone. - Small tables and low-cardinality columns often don't benefit — a scan can even be faster. - Don't guess whether an index is used. Run `EXPLAIN` (or `EXPLAIN ANALYZE`) and read the actual plan — Seq Scan vs Index Scan.

Curious what edge cases others have hit — especially around NULL handling and partial/expression indexes across different engines.

GITHUB README

````
# SQL Indexes: Finding 1 Row in a Million in ~20 Steps

How database indexes turn a slow full table scan into a fast lookup — the B-tree
behind them, when to use them, and the write/storage cost most tutorials skip.

## Problem

Find one row in a large table:

```sql
SELECT * FROM users WHERE email = 'kai@example.com';
````

With no index, the database performs a **full table scan** — it reads every row and checks each one. On a million-row table, that can mean a million checks. This scales linearly: double the rows, double the work.

Intuition

You can only skip data you haven't read if that data is **ordered**. If values are sorted, one comparison tells you which half to discard. Repeat, and you get binary search — each step halves the remaining rows, so the work grows logarithmically instead of linearly.

The table itself usually isn't sorted, so the database keeps a separate, small, sorted structure (a **B-tree**) that points to where each row lives — like the index at the back of a textbook.

Table size	Full scan (linear)	Index (logarithmic)
1,000	up to 1,000	~10
1,000,000	up to 1,000,000	~20
1,000,000,000	up to 1,000,000,000	~30

Approach

Create an index on the column you filter, sort, or join by:

```
CREATE INDEX idx_users_email ON users (email);
```

The engine now maintains a sorted B-tree of that column. A lookup starts at the root, follows the branch matching the value, and lands on the row in a handful of hops.
```

Indexes shine at: `WHERE` filters, `ORDER BY` (sort can be skipped), and `JOIN`s.

The trade-off: an index is a second copy that must stay in sync. Every `INSERT` / `UPDATE` / `DELETE` does extra work, and the index uses disk. **Reads get faster; writes get slower.**

Composite indexes (`CREATE INDEX ix ON orders (customer_id, created_at)`) only help queries using the leftmost column(s). Filtering on `created_at` alone won't use it.

Python Solution

A flat-list illustration of the same idea (databases do this in a B-tree):

```
from bisect import bisect_left


def full_table_scan(rows, target_email):
    """Brute force: check every row. O(n)."""
    for row in rows:
        if row["email"] == target_email:
            return row
    return None


def build_index(rows):
    """Like CREATE INDEX: a sorted (email, row) structure. O(n log n) once."""
    return sorted(((r["email"], r) for r in rows), key=lambda pair: pair[0])


def index_search(index, target_email):
    """Binary search the sorted index — the B-tree idea, flattened. O(log n)."""
    keys = [email for email, _ in index]
    pos = bisect_left(keys, target_email)
    if pos < len(index) and index[pos][0] == target_email:
        return index[pos][1]
    return None
```

Complexity

Operation	Time	Space
Full table scan	O(n)	O(1)
Index (B-tree)	O(log n)	O(n)
Write w/ index	O(log n) extra per row	—

Common Mistakes

- Indexing everything "just in case" — pure write overhead.
- `WHERE LOWER(email) = ...` — a function on the column disables the index.
- Wrong column order in a composite index.
- Guessing instead of running `EXPLAIN` (`Seq Scan` vs `Index Scan`).
- Indexing low-cardinality columns (booleans, small enums).

Video

▶️ Lesson 25 — *Find 1 Row in 1 Million in Just 20 Steps*: {LINK}

Article

Full written deep-dive (intuition, dry run, edge cases, interview questions) is part of the **Fun with Learning Technology** series.

Key Rule

> Index on purpose, not on impulse — and confirm with `EXPLAIN`. ```

Quick Review

When do you reach for a database index?

When filtering (WHERE), sorting (ORDER BY), or joining on a column becomes the query bottleneck as table rows grow large. Tell: repeated slow lookups on the same column.

Time complexity of index lookup vs full scan?

Indexed lookup is O(log n) via B-tree traversal; a full table scan is O(n) linear. On a million rows, ~20 steps vs a million.

Space cost of adding an index?

O(n) extra disk per index, plus slower INSERT/UPDATE/DELETE since every write must also update the index. Not free.

Common index bug or mistake?

Wrapping the column in a function (WHERE YEAR(date)=...) or leading wildcards (LIKE '%x') defeats the index, forcing a full scan. Also over-indexing every column.

Single-column vs composite index?

Composite indexes cover multi-column filters/sorts but only if the query uses a left-to-right prefix of the columns. Order matters; a skipped leading column wastes it.

Interview: why not index every column?

Each index adds storage and slows writes. Indexes help reads but tax every insert/update. Index only columns actually filtered, joined, or sorted frequently.

---

sql Lesson 25: Indexes and Query Performance (Database indexes are specialized data structures that provide a rapid lookup path to specific rows, dramatically reducing the amount of data the engine must scan. By minimizing I/O overhead, they transform slow linear scans into efficient logarithmic searches, which is critical for maintaining performance as tables grow to millions of records. You should reach for indexing when queries involving filtering, sorting, or joining become the primary bottleneck in your application's responsiveness.) — free Python cheat sheet